Menos prompts, mais ação

Desenvolvendo Agentes com Amazon Bedrock AgentCore

Carlos Oeiras



aws

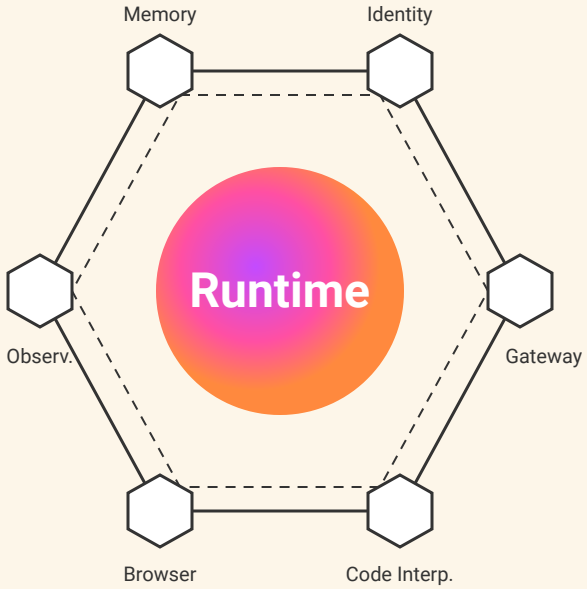
COMMUNITY DAY

BRASIL

EDIÇÃO NORTE · BELÉM/PA

aws
COMMUNITY DAY
BRASIL
EDIÇÃO NORTE · BELÉM/PA

Amazon Bedrock AgentCore



HTTP · MCP · A2A

contrato do contêiner, não do framework

Strands · LangGraph
CrewAI · qualquer LLM

Bedrock ou externo

Nesta demo:
só o Runtime

sem Memory, Gateway e Identity, de propósito

12 serviços modulares, adotáveis isoladamente: usar o Runtime não exige os demais · mapa completo no próximo slide

aws
COMMUNITY DAY
BRASIL
EDIÇÃO NORTE · BELÉM/PA

AgentCore: Build → Deploy → Assess

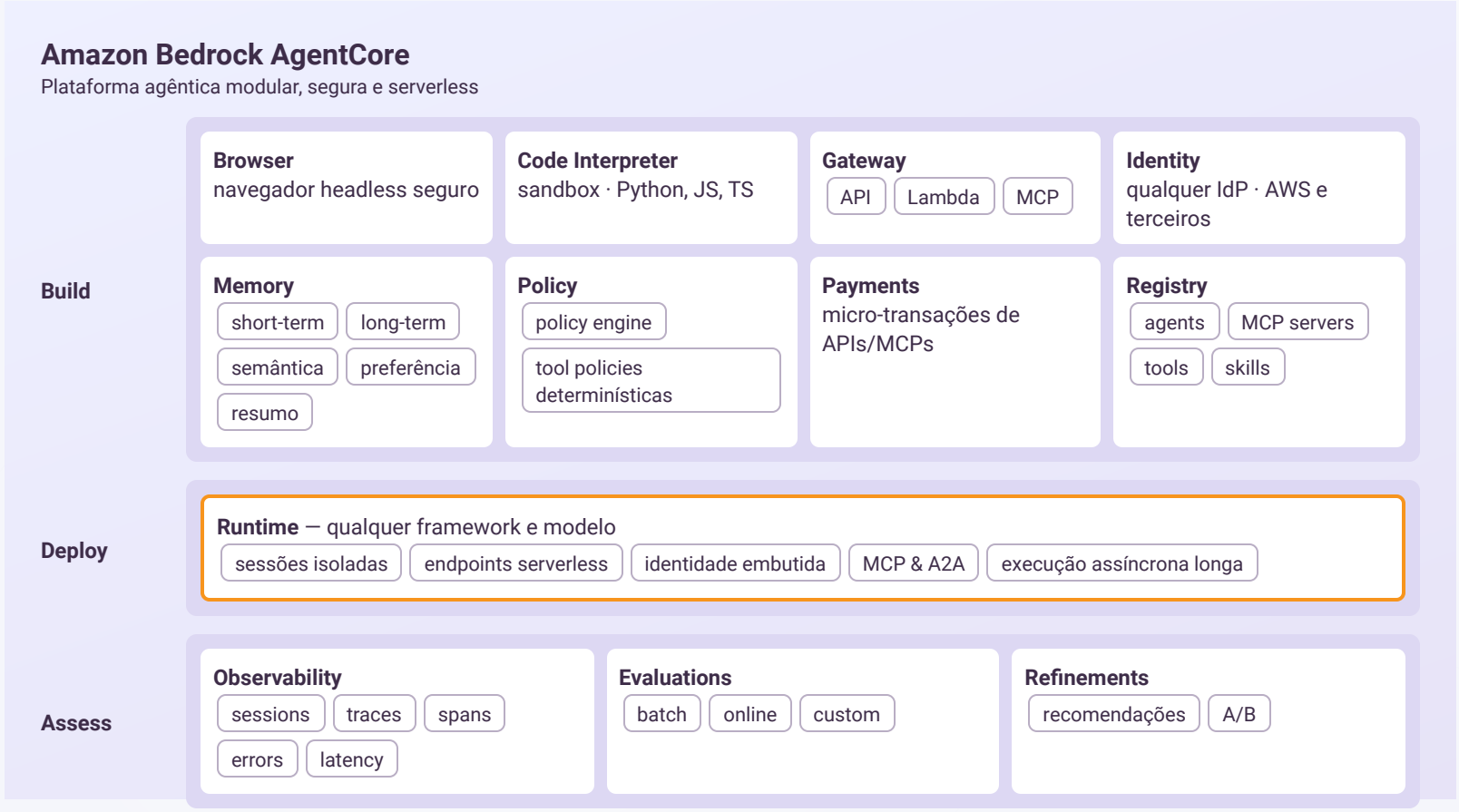


Diagrama adaptado do Amazon Bedrock AgentCore Developer Guide

Build 8 serviços

Browser · Code Interpreter · Gateway · Identity · Memory · Policy · Payments · Registry.
Cada um é uma API própria, com IAM e cobrança separados.

Deploy NA DEMO

Runtime: sessão = microVM dedicada; endpoints serverless; MCP e A2A.
Cobrança por CPU/memória ativa, não por instância ligada.

Assess

Observability · Evaluations · Refinements.
Só existe trace do que foi instrumentado (OTEL/ADOT).

aws COMMUNITY DAY BRASIL
EDIÇÃO NORTE · BELÉM/PA

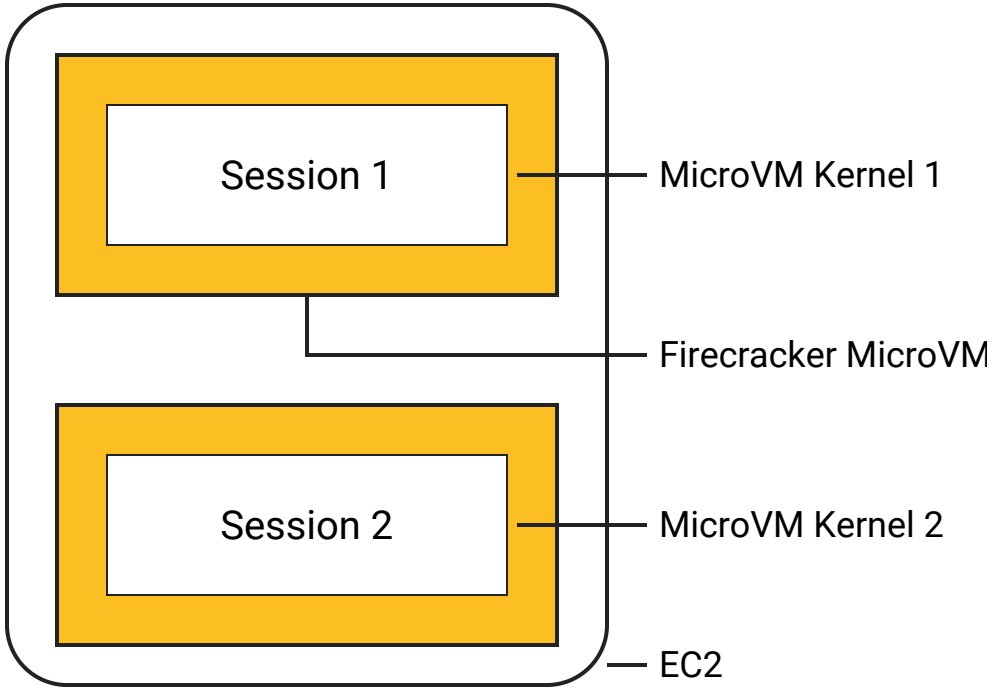
Runtime: isolamento por sessão

microVM

Firecracker: CPU, memória e filesystem dedicados por sessão

1 ID

`runtimeSessionId` ≥ 33 caracteres, gerado pelo cliente. Reusar = mesmo contexto; **vazar = risco**.



8 h

`maxLifetime` = 28800 s: teto rígido; tarefa maior exige checkpoint externo

15 min

idle = 900 s; `/ping` com `HealthyBusy` impede o encerramento (tarefas assíncronas)

Estado em memória morre com a microVM. Durável → AgentCore Memory ou store próprio.

aws
COMMUNITY DAY
BRASIL
EDIÇÃO NORTE · BELÉM/PA

Onde a ferramenta entra



O LLM não executa nada: emite `toolUse` (nome + args JSON validados pelo schema); o loop do agente executa.

`toolResult` volta pareado por `toolUseId`; cada volta do loop = nova chamada ao modelo (+latência, +tokens).

Fluxo ilustrativo baseado na documentação AWS; sem medições de tempo desta demo.

aws
COMMUNITY DAY
BRASIL
EDIÇÃO NORTE · BELÉM/PA

Demo: checkout indisponível

Serviço simulado	Estado inicial
checkout-api	**DOWN**
payment-api e order-api	HEALTHY

Estado em dicionário em memória, dentro da microVM da sessão

Leitura: get_services · get_service_status · get_service_metrics

Escrita: restart_service (mock, efeito colateral)

Nova sessão = checkout-api DOWN de novo.

aws
COMMUNITY DAY
BRASIL
EDIÇÃO NORTE · BELÉM/PA

Do Docker ao AgentCore

Código

Strands Agent + 4 `@tool` (schema vem de type hints + docstring); FastAPI em `0.0.0.0:8080`

Contêiner

Obrigatório **linux/arm64** · `GET /ping` → `{"status":"Healthy"}` · `POST /invocations`

ECR

`docker buildx --platform linux/arm64 --push` → ECR privado

Runtime

`CreateAgentRuntime` → V1 + endpoint DEFAULT. Role: trust `bedrock-agentcore.amazonaws.com`, ECR pull, `bedrock:InvokeModel` + **InvokeModelWithResponseStream**, Logs/X-Ray

aws
COMMUNITY DAY
BRASIL
EDIÇÃO NORTE · BELÉM/PA

Demo ao vivo

Consultar

“Quais serviços existem?”

esperado: 1 toolUse → get_services

Investigar

“O checkout-api está com problema? Consulte status e métricas.”

esperado: get_service_status + get_service_metrics; o modelo decide a ordem

Agir e verificar

“Resolva o problema do checkout-api e verifique o resultado.”

esperado: restart_service **e depois** get_service_status · “resolvido” sem nova leitura = verificação alucinada

`InvokeAgentRuntime(agentRuntimeArn, qualifier=DEFAULT, runtimeSessionId)` · **mesma sessão · DOWN → HEALTHY**

aws
COMMUNITY DAY
BRASIL
EDIÇÃO NORTE · BELÉM/PA

A execução deixa evidências

Session ID: demo-checkout-…(≥33 chars)

Session summary

Traces: **3** (1 por prompt)

Agent: **ops-agent** · Runtime V1 · DEFAULT

O que procurar

- span `execute_tool` por chamada
- status do span = OK (toolResult sem erro)
- trace 3: restart **antes** do status
- tokens e latência por `chat`

Trace 3 — estrutura esperada dos spans

```
▾ invoke_agent Strands Agents
  ▾ execute_event_loop_cycle
      chat → toolUse restart_service
      execute_tool restart_service {"service":"checkout-api"}
  ▾ execute_event_loop_cycle
      chat → toolUse get_service_status
      execute_tool get_service_status → HEALTHY
  ▾ execute_event_loop_cycle
      chat → resposta final
```

Ilustrativo — substituir por print real da demo. Requer ADOT no contêiner (`opentelemetry-instrument`) + Transaction Search; sem isso, o console fica vazio.

aws
COMMUNITY DAY
BRASIL
EDIÇÃO NORTE · BELÉM/PA

O que conferir na demo

- **toolUse certo, args válidos e toolResult sem erro?**
- **Ação autorizada (IAM / Policy)?** na demo a role é ampla — proposital
- **Estado confirmado por nova leitura, não pela palavra do modelo?**

Gateway | Policy | Identity | Observability

Em produção: Gateway expõe as tools, Policy autoriza cada chamada,
Identity autentica quem age e Observability comprova.

Dúvidas?

Qual ferramenta você conectaria primeiro?

Carlos Oeiras
linkedin.com/in/carlosoeiras

Newsletter Além do Óbvio

